

mucho ánimo estudiando

- Búsquedas Secuenciales

Búsqueda Lineal Primaria $O(n)$

[Linear Search and Binary Search Animated \(youtube.com\)](#)

```
int bus_primaria(Tdef x, TVector v, int ini, int fin) {  
    for (int pos = ini; pos <= fin; pos++) {  
        if (v[pos] == x)  
            return pos;  
    }  
    return -1;  
}
```

Movimiento al frente: colocar en primer lugar cada elemento buscado [listas]

Transposición: intercambiar cada elemento buscado con el anterior (si no es el primero) [arrays]

Búsqueda Binaria $O(\log_2 n)$

como un diccionario (tiene que estar ordenado ya)

```
int bus_binaria2(Tdef x, TVector v, int ini, int fin){  
    pos = ini;  
    while (ini < fin) {  
        pos = (ini + fin) / 2;  
        if (v[pos] == x)  
            return pos;  
        else  
            if (v[pos] < x)  
                ini = pos + 1;  
            else  
                fin = pos - 1;  
    }  
    if (v[ini] == x)  
        return ini;  
    return -1;  
}
```

- Tablas Hash:

Funciones de dispersión:

Truncamiento, se ignora parte de la clave

Módulo de truncamiento, usamos parte de la secuencia Módulo

Suma, se suman los dígitos en ascii

Plegado por desplazamientos, se parten en n cifras y se suman

Plegado por las fronteras, se intercala la suma entre n números normales e inversos

Centro del cuadrado, se hace el cuadrado y se toman unos dígitos específicos

Pseudo-aleatorio

Colisiones: coinciden en clave, son sinónimos

Dispersión abierta: no hay límite en las celdas de la tabla

Dispersión cerrada: hay límite en las celdas de la tabla, puede haber desbordamiento

Si hay desbordamiento, se determina otro lugar por función de exploración:

Exploración lineal, por contigüidad (+1,+2,+3,+4)

Exploración cuadrática, se busca el cuadrado del anterior (+1,+4,+9,+16)

Dispersión doble, se usa otra función de dispersión

Redispersión, se usa varias veces una función de dispersión

- ORDENACIÓN

Ordenación por inserción: $O(n^2)$

[Insertion Sort Algorithm Tutorial \(youtube.com\)](#)

```
void insertar(int sec[], int i, int x) {  
    int j = i - 1;  
    while (j >= 0 && sec[j] > x) {  
        sec[j + 1] = sec[j];  
        j--;  
    }  
    sec[j + 1] = x;
```

```
void ordenacionPorInsercion(int sec[], int n) {  
    for (int i = 1; i < n; i++) {  
        insertar(sec, i, sec[i]);  
    }  
}
```

BinSort , como solución al anterior con una búsqueda binaria $O(n^2)$

```
void binSort(int sec[], int n) {
    int ini, fin, med, x, j;
    for (int i = 1; i < n; i++) {
        j = i;
        x = sec[i];
        ini = 0;
        fin = i - 1;
        while (ini <= fin) {
            med = (ini + fin) / 2;
            if (sec[med] < x)
                ini = med + 1;
            else
                fin = med - 1;
        }
        for (j = i - 1; j >= ini; j--)
            sec[j + 1] = sec[j];
        sec[ini] = x;
    }
}
```

Ordenación por Selección $O(n^2)$ [Selection Sort Algorithm Tutorial \(youtube.com\)](https://www.youtube.com/watch?v=Uj4333Uu2h4)

```
void selectionSort(std::vector<int>& seq) {
    int size = seq.size();
    for (int i = 0; i < size - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < size; ++j) {
            if (seq[j] < seq[minIndex]) {
                minIndex = j;
            }
        }
        // Intercambiar elementos
        int temp = seq[minIndex];
        seq[minIndex] = seq[i];
        seq[i] = temp;
    }
}
```

Ordenación por intercambio Burbuja $O(n^2)$ [Bubble Sort | Animation | Coddick \(youtube.com\)](https://www.youtube.com/watch?v=Uj4333Uu2h4)

```
void bubbleSort(std::vector<int>& seq) {
    int n = seq.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (seq[j] > seq[j + 1]) {
                std::swap(seq[j], seq[j + 1]);
            }
        }
    }
}
```

Ordenación por intercambio Sacudida Shake $O(n^2)$

```
void shakeSort(std::vector<int>& sec) { \(2\) Shaker sort - YouTube
    int n = sec.size();
    int ini = 1;
    int fin = n - 1;
    int cam = n;

    while (ini < fin) {
        for (int j = fin; j >= ini; j--) {
            if (sec[j] < sec[j - 1]) {
                std::swap(sec[j - 1], sec[j]);
                cam = j;
            }
        }
        ini = cam + 1;
        for (int j = ini; j <= fin; j++) {
            if (sec[j] < sec[j - 1]) {
                std::swap(sec[j - 1], sec[j]);
                cam = j;
            }
        }
        fin = cam - 1;
    }
}
```

Ordenación HeapSort

$O(n \log n)$

[\(2\) HEAP SORT | Sorting Algorithms | DSA | GeeksforGeeks - YouTube](#)

```
void baja( int i; Tvector &sec; int n ) {
    while ( 2*i <= n ){
        h1 = 2*i;
        h2 = h1 + 1;
        if (h1 == n)
            h = h1
        else if (sec[h1] > sec[h2])
            h = h1
        else h = h2;
        if (sec[h] <= sec[i])
            break;
        else {
            swap(sec[i],sec[h]);
            i = h;
        }
    }
};

void heapsort( Tvector sec; int n ) {
    for (int i = n/2; i > 0; i--)
        baja(i, sec, n);
    for (int i = n; i > 1; i--) {
        swap(sec[1],sec[i]);
        baja(1,sec,i-1);
    }
}
```

Ordenación QuickSort

$O(n^2)$ en el peor caso pero es $O(n \log n)$ en el caso medio.

[QUICK SORT | Sorting Algorithms | DSA | GeeksforGeeks \(youtube.com\)](#)

```
void Qsort( sec, ini, fin) {
    i = ini; f = fin;
    p = sec[(i+f)/2];
    while (i <= f){
        while (sec[i] < p) i++;
        while (sec[f] > p) f--;
        if (i <= f) {
            swap(sec[i],sec[f]);
            i++;
            f--;
        }
    };
};

if (ini < f) Qsort(sec, ini, f);
if (i < fin) Qsort(sec, i, fin);
```

}

Ordenación MergeSort [Ordenar por combinación en 3 minutos \(youtube.com\)](https://www.youtube.com/watch?v=3m3m3m3m3)

```
void merge(std::vector<int>& sec, int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    std::vector<int> L(n1), R(n2);
```

```
    for (int i = 0; i < n1; i++)
```

```
        L[i] = sec[left + i];
```

```
    for (int j = 0; j < n2; j++)
```

```
        R[j] = sec[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            sec[k] = L[i];
```

```
            i++;
```

```
        } else {
```

```
            sec[k] = R[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```
    while (i < n1) {
```

```
        sec[k] = L[i];
```

```
        i++;
```

```
        k++;
```

```
    }
```

```
    while (j < n2) {
```

```
        sec[k] = R[j];
```

```
        j++;
```

```
        k++;
```

```
    }
```

```
}
```

```
mergesort (array a)
```

```
    if ( n == 1 )
```

```
        return a
```

```
    arrayOne = a[0] ... a[n/2]
```

```
    arrayTwo = a[n/2+1] ... a[n]
```

```
    arrayOne = mergesort ( arrayOne )
```

```
    arrayTwo = mergesort ( arrayTwo )
```

```
    return merge ( arrayOne, arrayTwo )
```

```
merge ( array a, array b )
```

```
    array c
```

```
    while ( a and b have elements )
```

```
        if ( a[0] > b[0] )
```

```
            add b[0] to the end of c
```

```
            remove b[0] from b
```

```
        else
```

```
            add a[0] to the end of c
```

```
            remove a[0] from a
```

```
    // At this point either a or b is empty
```

```
    while ( a has elements )
```

```
        add a[0] to the end of c
```

```
        remove a[0] from a
```

```
    while ( b has elements )
```

```
        add b[0] to the end of c
```

```
        remove b[0] from b
```

```
    return c
```

También pueden entrar ShellSort, y RadixSort peeeero , solo aprendete la metodología

[Shell Sort \(youtube.com\)](https://www.youtube.com/watch?v=3m3m3m3m3)

[Radix Sort Animations | Data Structure | Visual How \(youtube.com\)](https://www.youtube.com/watch?v=3m3m3m3m3)

⚠️✅ **TODOS LOS ALGORITMOS** ⚠️✅ **AYEDA** ⚠️✅ **@victor_rgez**

si te gustaron estos apuntes tengo más en mi perfil.
Todos gratis, pero ayuda mucho un follow y que descargues algunos.

mucho ánimo estudiando